

Algorithm Final Note

김성열 교수님

2021년 2학기

차 례

차 례	1
1 Basic	2
2 Minimum Spanning Tree	5
3 Shortest Path	9
4 Deadline Scheduling, Tape Storage	13
5 Merge Sort	16
6 Quick Sort	17
7 Approximate Median Problem	19
8 Closest Pair	21
9 Convex Hull	24
10 Dynamic Programming	26
11 Graph Traversal	31
12 Cut Vertex	33
13 DAG (Directed Acyclic Graph)	35
14 Exhaustive Search	38
15 Randomized Algorithm	39

1 Basic

1.1 수학적 귀납법

$P(1)$ 이 참이고, $P(n-1) \rightarrow P(n)$ 이 참이면 $P(n)$ 은 모든 자연수 n 에 대해서 참이다.

$P(n-1)$ 이 참이라고 가정하는 이유: $P(n-1)$ 이 거짓이면 $P(n-1) \rightarrow P(n)$ 전체가 Vacuously True 이므로 할 말이 없다.

1.2 정렬

정렬의 시간복잡도는 $O(n \log n)$ 보다 빠를 수 없다. 정렬 알고리즘의 정당성 증명: a 배열이 입력이라고 가정하고 정렬이 완료된 배열을 b 라고 하면 a, b 배열의 값의 같고 b 배열의 값들은 $b[0] < b[1] < \dots < b[n-1]$ 과 같이 오름차순으로 정렬되어 있어야 한다.

Selection Sort (선택 정렬)

1. 집합 조건을 깰 수 있는 코드는 없다.
2. $n = 1$ 인 경우: 할 일이 없다.
3. $n - 1$ 일 때 sort 함수가 성공한다고 가정하자. (sort() 함수가 실패할 경우는 할 말이 없다.)
즉, 재귀 호출이 끝난 후 $a[1] < a[2] < \dots < a[n-1]$ 이다.
4. $a[0]$ 에는 항상 배열 전체에서 가장 작은 값이 들어가있다. 따라서 재귀로 들어가기 전부터 $\forall x((x > 0) \rightarrow (a[0] < a[x]))$ 가 성립했다. 그러므로 n 일 때 $a[0] < a[1] < a[2] < \dots < a[n-1]$ 이므로 sort 함수가 성공한다.
5. 시간복잡도: $T(n) = n + T(n-1) = n + (n-1) + T(n-2) = O(n^2)$

Merge Sort

1. 집합 조건을 깰 수 있는 코드는 없다.
2. $n = 1$ 인 경우: 할 일이 없다.
3. $n/2$ 일 때 sort() 함수가 성공한다고 가정하자. (sort() 함수가 실패할 경우는 할 말이 없다.)
즉, 재귀 호출이 끝난 후 새로 생성된 배열 c 에 대하여 $c[0] < c[1] < \dots < c[n/2-1]$ 이고, $c[n/2] < c[n/2+1] < \dots < c[n-1]$ 이다.
4. n 일때는 merge 알고리즘의 정확성 (집합관계가 깨지지 않고, sorted 된 상태로 merge 됨) 에 의해 sort() 함수가 성공한다.
5. 시간복잡도: $T(n) = n + 2T(n/2) = n + 2(n/2 + 2T(n/4)) = \dots = O(n \log n)$

1.3 자료구조의 정의

1. Array: 연속된 메모리에 동일한 타입의 데이터들이 저장됨. 정렬되어있으면 검색이 빠르지만, 일반적으로는 느림. 삽입 삭제도 느림
2. Linked List: 데이터와 다음 노드의 주소가 들어간 Node들의 리스트. Array와 다르게 개수 제한이 없다.
3. Binary Search Tree: 하나의 Root Node가 있고, Root Node 에는 두 개의 자식 노드가 존재한다. 각 자식 노드들은 다시 subtree의 Root Node가 된다.
4. Heap: 우선순위 큐를 만들기 위해 사용. depth: $\log n$

1.4 기출문제 Set 1

기출문제 1-1

자연수 x 에 대하여 Recursive Sum 함수 `int sum(int x) ... return x+sum(x-1);` 에서 이 함수가 1에서 x 까지의 합을 계산함을 수학적 귀납법을 이용해 증명하라. (17중간)

$\text{sum}(1)$ 이 리턴하는 값은 1이므로 1부터 1까지의 합을 계산하였다. $n-1$ 일 때 $\text{sum}(n-1)$ 이 리턴하는 값이 $1 + 2 + \dots + x - 1$ 이라고 가정하자. 코드에서 $\text{sum}(n)$ 은 $n + \text{sum}(n-1)$ 을 리턴하는데 $\text{sum}(n-1)$ 은 $1 + 2 + \dots + x - 1$ 이므로 $n + \text{sum}(n-1)$ 은 $1 + 2 + \dots + x$ 이다. 따라서 $\text{sum}(x)$ 함수는 1에서 x 까지의 합을 계산한다.

기출문제 1-2

모든 자연수가 같다는 것을 증명하는 과정이다. 두 자연수 a, b 가 주어졌을때 증명이 옳으면 5를, 아니면 번호와 틀린 이유를 써라. ($n = \max(a, b)$) (17중간)

- 1) $n=1$ 일 때, $\max(a, b)=1$ 이고, $a=b=1$ 이다.
- 2) $n=\max(a, b)=k$ 일때, $a=b$ 라고 가정하자
- 3) $n=\max(a, b)=k+1$ 일때, $\max(a-1, b-1)=k$ 이다.
- 4) $\max(a-1, b-1)=k$ 이므로, $a-1=b-1$ 이다.
- 5) 따라서 모든 자연수는 같다.

기출문제 1-3

Graph와 Tree의 정의를 제시하시오. (Tree 는 Rooted Tree 가 아님) (15중간)

Graph는 Vertex들의 집합 V 와 두 개의 Vertex 사이를 잇는 Edge들의 집합 E 로 이루어진 자료구조이다. Tree는 사이클이 존재하지 않는 Graph를 의미한다.

기출문제 1-4

수학적 귀납법을 이용하여 2 이상인 모든 자연수 n 에 대해서 $n^2 > n$ 임을 증명하라. 단, 증명에는 다음의 사실들만을 사용할 수 있다. (12중간)

- a) 부등식의 기본 성질
- b) 모든 자연수 a 에 대해서 $2a > 0$ 이 성립.
- c) 모든 자연수 a 에 대해서 $a + 1 > a$ 가 성립. ($3 > 2, 4 > 3, 4 > 2$ 등도 모두)
- d) 모든 자연수 a, b 에 대해서 $a > b$ 는 $a - b > 0$ 과 같은 의미.
- e) 모든 자연수 a 에 대해서 $(a + 1)^2 = a^2 + 2a + 1$

2 Minimum Spanning Tree

2.1 Prim 알고리즘

MST

Spanning Tree는 그래프의 모든 정점과 간선의 부분집합으로 구성된 부분 그래프이다. MST는 주어진 그래프의 연결 그래프들 중 엣지들의 비용이 최소가 되도록 하는 그래프를 의미한다. (단, 사이클을 생성하지 않아야 한다.)

Prim 알고리즘의 정당성 증명

- Edge Set인 T_{mst} 는 정답 중 하나라고 가정
- 알고리즘이 끝난 후 Prim 알고리즘이 가지고 있는 edge set T 는 T_{mst} 가 될 것이다.
- ※ 주의사항: T_{mst} 는 증명 중간에 바뀔 수 있기 때문에 미리 잡고 시작할 수 없다. 하지만 정답이라는 사실은 항상 유지된다!

Proof. $\exists T_{mst} \rightarrow T \subseteq T_{mst}$: "Prim Algorithm이 가지고 있는 edge set T 는 어떤 정답의 부분집합이다."

1. Base: 알고리즘 시작 시 node 1개, edge 0개이므로 $T = \emptyset$ 이다. 따라서, $T \subseteq T_{mst}$ 가 성립한다.
2. Step: 어떤 시점에 $T \subseteq T_{mst}$ 라고 가정하자. 이후 한 스텝을 진행해서 Prim Algorithm이 edge e_1 을 추가하고 현재 edge set 은 $T' = T \cup \{e_1\}$ 이다.
 - Case 1: $T' \subseteq T_{mst}$ 이면 명제 성립하고 끝!
 - Case 2: $T' \not\subseteq T_{mst}$ 이면 반드시 사이클이 생성된다. 이 경우 사이클 내부에 $(e \neq e_1) \wedge (e \notin T') \wedge (T \cup \{e\} \text{ is not a cycle}) \wedge (e \text{ is connected to } T)$ 를 만족하는 edge e 가 반드시 존재한다.
3. 위의 Case 2 에서 $W(e_1)$ 과 $W(e)$ 의 대소관계에 따른 케이스를 나눠서 생각해보자.
 - Case 1: $W(e_1) > W(e)$ 이면 알고리즘은 항상 작은 것만 넣어야 하는데 더 작은 것이 존재한다는 뜻이므로 알고리즘에 모순된다.
 - Case 2: $W(e_1) < W(e)$ 이면 $(T_{mst} - \{e\}) \cup \{e_1\}$ 인 더 좋은 답이 존재하게 되므로 T_{mst} 의 정의에 모순된다.
 - Case 3: $W(e_1) = W(e)$ 이면 $(T_{mst} - \{e\}) \cup \{e_1\}$ 이것도 정답이 된다. 이 집합을 T'_{mst} 라고 하면 $T' \subseteq T'_{mst}$ 이므로 명제가 성립한다. 증명 끝!!

□

Prim 알고리즘의 시간복잡도

정점 개수 n , 엣지 개수 m 이라고 하자.

- Graph 표현: Linked List 로 구현. 각 노드에는 연결된 엣지들이 $(goalnode, weight)$ 형태로 저장되어 있다. (사용 시간: m)
- 노드가 트리에 들어갔는지 마킹: 노드 길이만큼 일반 배열 생성해서 구현. boolean 값으로 몇 번 노드가 있는지 없는지만 확인하는 용도이다. 중복과 사이클 여부를 판별할 때도 사용 가능하다. (사용 시간: edge 하나 꺼낼때마다 2칸씩 확인하므로 총 $4n$)
- Edge 저장소: Priority Queue로 구현. 노드에 연결된 엣지들은 $(node1, node2, weight)$ 형태로 저장한다. (사용 시간: 모든 edge들이 2번씩 넣고 빠지고, 우선순위 큐를 사용하므로 총 $2m \log n$)
- Edge의 개수가 $n - 1$ 이면 종료한다.
- 시간복잡도: $O(m \log n)$

2.2 Kruskal 알고리즘

Kruskal 알고리즘의 정당성 증명

Proof. $\exists T_{mst} \rightarrow T \subseteq T_{mst}$: "Kruskal Algorithm이 가지고 있는 edge set T 는 어떤 정답의 부분집합이다."

1. Base: 알고리즘 시작 시 node 1개, edge 0개이므로 $T = \emptyset$ 이다. 따라서, $T \subseteq T_{mst}$ 가 항상 성립한다.
2. Step: 어떤 시점에 $T \subseteq T_{mst}$ 라고 가정하자. 이후 한 스텝을 진행해서 Kruskal Algorithm이 edge e 를 추가하고 현재 edge set 은 $T' = T \cup \{e\}$ 이다.
 - Case 1: $T' \subseteq T_{mst}$ 이면 명제 성립하고 끝!
 - Case 2: $T' \not\subseteq T_{mst}$ 이면 반드시 사이클이 생성된다. 이 경우 사이클 내부에 $(e_1 \neq e) \wedge (e_1 \notin T)$ 를 만족하는 edge e_1 이 반드시 존재한다.
3. 위의 Case 2 에서 $W(e_1)$ 과 $W(e)$ 의 대소관계에 따른 케이스를 나눠서 생각해보자.
 - Case 1: $W(e_1) > W(e)$ 이면 $(T_{mst} - \{e\}) \cup \{e_1\}$ 인 더 좋은 답이 존재하게 되므로 T_{mst} 의 정의에 모순된다.
 - Case 2: $W(e_1) < W(e)$ 이면 Kruskal Algorithm 에 의해 e_1 이 먼저 추가되었어야 한다. 따라서 알고리즘에 모순된다.
 - Case 3: $W(e_1) = W(e)$ 이면 $(T_{mst} - \{e\}) \cup \{e_1\}$ 이것도 정답이 된다. 이 집합을 T'_{mst} 라고 하면 $T' \subseteq T'_{mst}$ 이므로 명제가 성립한다. 증명 끝!!

□

Kruskal 알고리즘의 구현과 시간 복잡도

- Edge 저장소: weight에 따라 정렬해서 저장할 배열 생성해서 구현. (사용 시간: $m \log m$)
- Union Find: 노드가 연결되어 있는지 확인하는 용도. 연결되어 있지 않으면 Edge를 추가하고, 연결되어 있으면 그 Edge는 버린다. (사용 시간: n)
- 시간 복잡도: $O(m \log m)$

2.3 Prim and Kruskal can find any solution

Prim Algorithm 과 Kruskal Algorithm 은 항상 모든 솔루션을 찾을 수 있다. 알고리즘의 정답을 여러가지 답들 중 아무거나 T_{mst} 하나로 고정하자. 이제 알고리즘이 이 답을 찾을 수 있음을 보이면 된다.

알고리즘이 T 까지 만든 상태에서 T_{mst} 에 속하지 않는 Edge e 를 넣었다면, $T \cup \{e\}$ 는 반드시 Cycle이 된다. 이때 Cycle 내에 다른 엣지 e_1 이 존재하고, 반드시 $w(e_1) = w(e)$ 를 만족한다. 즉 e_1 도 원래 이번 스텝에서 고려 대상이었고, 이 때 e_1 을 추가했다면 T_{mst} 에 속하게 되었을 것이다. 따라서 Prim 알고리즘은 반드시 T_{mst} 를 찾을 수 있다. 마찬가지로 Kruskal 알고리즘도 모든 Solution을 반드시 찾아낼 수 있다.

이 사실을 이용하여 "그래프의 모든 엣지들의 weight 가 다를 경우 답은 유일함" 을 증명할 수 있다. Prim, Kruskal 은 $W(e_1) = W(e)$ 인 경우에만 답을 여러개 구할 수 있다. 모든 Edge들의 weight 가 다르다면 이런 경우가 존재하지 않으므로, T_{mst} 에 속하지 않는 Edge e 자체가 존재하지 않는다. 따라서 답은 유일하게 T_{mst} 하나로만 정해진다.

2.4 기출문제 Set 2

기출문제 2-1

모든 Weight가 양수인 연결된 무향 그래프에서 엣지의 Weight가 모두 다르다고 가정하면 Minimum Spanning Tree는 단 하나 밖에 없음을 증명하라. (12 중간, 17중간) / Minimum Spanning Tree 문제에서 주어진 입력에 대한 답이 여러 개인지 여부를 판단하는 알고리즘을 제시하고 그 정확성을 세 줄 이내로 간략히 설명하라. (13중간)

MST가 2개 존재한다고 가정한 귀류법 풀이: (증명 보러가기)

교수님 방식: 2.3절 참고

기출문제 2-2

MST에서 Union-Find가 어떻게 사용되는지 설명하시오. (15중간) / Minimum Spanning Tree를 푸는 Kruskal 알고리즘에서 특정 엣지를 추가했을 때 cycle이 생기는지 판별하는 효율적인 방법은 무엇인지 설명하라. (14중간)

알고리즘에서는 MST의 정답 집합 T 를 찾기 위해 먼저 엣지들을 가중치에 따라 정렬한다. 각 엣지에 연결된 정점들이 같은 덩어리인지 Union-Find로 확인하고, 같은 덩어리가 아니면 Union-Find에 한 덩어리로 추가하고 edge를 정답 집합 T 에 추가한다. 같은 덩어리일 경우 해당 엣지는 버린다.

3 Shortest Path

3.1 Shortest Path

Dijkstra Algorithm

Dijkstra Algorithm

0. Terminology

- $G = (V, E)$: 그래프 입력 - 정점과 엣지들
- n : 노드의 총 개수
- v_0 : Source Node
- d_{min} : S.P. length (출력될 정답)
- $g(v, u)$: v 에서 u 로 가는 Edge Weight
- R : Red Set. 여기 속한 노드들의 $d_{min}(v)$ 는 최종 정답을 의미함
- B : Blue Set. 여기 속한 노드들의 $d_{min}(w)$ 는 Red Set만 거치는 S.P.의 길이를 의미함
- Input: $Dijkstra(G, n, v_0, d_{min})$

1. Source와 연결된 모든 정점마다 Edge Weight 를 $d_{min}(v)$ 에 넣어 갱신한다. Source와 연결되지 않은 노드들의 Edge Weight는 ∞ 라고 가정한다.

2. R 에 Source 노드를 넣는다. Red 집합의 크기가 n 이 될때까지 다음 과정(3번)을 반복한다.

3. R 바깥에 있는 $d_{min}(u)$ 들 중 가장 작은 값을 가진 노드 u 를 찾아서 R 에 넣는다. 그리고 R 에 속하지 않는 노드들 w 에 대해서 $\min(d_{min}(w), d_{min}(u) + g(u, w))$ 값으로 집합 B 를 갱신시켜준다.

S.P. Length for each Node

Proof. Dijkstra Algorithm이 각 노드들의 S.P.의 길이를 찾을 수 있다.

1. Base: $n = 1$ 즉, Source 노드 하나만 존재하므로 $d_{min}(v_0) = 0$ 으로 주장이 성립한다.
2. Step: 어떤 시점에 집합 $R = v_0, v_1, \dots, v_i$, $B = w_0, w_1, \dots, w_j$ 이라고 하자. B 에 속한 노드들 중 $d_{min}(w)$ 값이 가장 작은 노드를 w 라고 하자. 이 노드를 집합 R 에 추가하자.

집합 B 에 속하는 노드들 중 $d_{min}(w)$ 가 제일 작은 것을 넣는 것이 정당한 이유

w 가 다음에 집합 R 에 추가할 노드가 아니라면, R 집합을 거쳐서 오는 제일 짧은 것이 아니라는 뜻이므로 B 집합의 노드를 거쳐서 와야 더 짧아진다. 그 경로에서 처음으로 만난 B 집합의 노드는 반드시 존재하는데, 그 노드로의 경로는 $d_{min}(w)$ 보다 항상 크다. 따라서 모순이 발생하므로, w 를 넣는 것이 정당하다.

3. R 집합에 B 에 있던 노드 u 를 추가하면, u 를 통하는 더 좋은 길이 생길 수 있기 때문에 집합 B 의 $d_{min}(w)$ 값들을 모두 $\min(d_{min}(w), d_{min}(u) + g(u, w))$ 값으로 갱신해줘야 한다.

$u \rightarrow w$ 인 경우만 따지는 것이 정당한 이유 (중간에 다른 노드를 거치지 않는 이유)

$u \rightarrow v \rightarrow w$ 인 경우가 w 까지의 S.P. 이라고 가정해보자. u 는 방금 집합 R 에 들어갔기 때문에, v 는 그 이전에 집합 R 에 들어가있었다. 즉, v 가 R 에 들어간 시점에는 u 가 R 바깥에 있었다. 이 경우, source에서 v 로 가는 S.P. 중에서 u 를 안쓰고 R 에 속하는 경로가 존재하게 된다. 따라서 u 가 필요 없어지므로 가정에 모순이다. 그러므로 $u \rightarrow w$ 인 경우만 따지는 것이 정당하다.

□

Actual S.P. for each Node

Proof. Dijkstra Algorithm을 통해 각 노드들의 실제 S.P.를 찾을 수 있는 S.P. Tree를 만들 수 있다.

모든 노드는 최단경로 내에서 자기 앞에 무슨 노드가 있는지 알고 있다고 가정하자. R 집합에 B 에 있던 노드 u 를 추가한 후, 현재 B 에 있는 노드 w 까지의 기존 S.P.와 $u \rightarrow w$ 를 통과하는 새로운 경로 중에서 더 짧은 것으로 경로가 갱신된다. 거리가 같을 경우, $u \rightarrow w$ 의 새로운 경로를 버리고 기존 S.P.를 선택한다. 따라서 S.P. Tree가 만들어진다.

□

All possible S.P. for each Node

Proof. Dijkstra Algorithm을 통해 각 노드들의 모든 가능한 S.P. 들을 찾을 수 있다.

모든 노드는 최단경로 내에서 자기 앞에 무슨 노드가 있는지 알고 있다고 가정하자. R 집합에 B 에 있던 노드 u 를 추가할 때, 현재 B 에 있는 노드 w 까지의 기존 S.P.와 $u \rightarrow w$ 를 통과하는 새로운 경로 중에서 더 짧은 것으로 경로가 갱신된다. 거리가 같을 경우, 두 노드의 정보를 모두 저장하면 된다. 따라서 S.P. DAG(Directed Acyclic Graph)가 만들어진다.

□

Dijkstra Algorithm's Implementation and Time Complexity

1. $|V| = n$, $|E| = m$ 이라 하자.
2. Source와 연결된 모든 정점마다 Edge Weight 를 $d_{min}(v)$ 에 넣어 갱신한다. Source와 연결되지 않은 노드들의 Edge Weight는 ∞ 라고 가정한다. - $O(n)$
3. R 에 Source 노드를 넣는다. - $O(1)$
4. R 바깥에 있는 $d_{min}(u)$ 들 중 가장 작은 값을 가진 노드 u 를 찾아서 R 에 넣는다. 그리고 R 에 속하지 않는 노드들 w 에 대해서 $\min(d_{min}(w), d_{min}(u) + g(u, w))$ 값으로 집합 B 를 갱신시켜 준다.
 - 가장 작은 $d_{min}(u)$ 는 Priority Queue를 사용하여 검색하므로 $O(\log n)$ 이 걸린다. 그리고 갱신할 때는 u 와 인접한 노드들만 따지면 된다. 갱신하는 것도 $O(\log n)$ 이 걸리고, 이 행위를 모든 Edge들에 해야하므로 총 m 번 일어난다. 따라서 $O(m \log n)$ 의 시간이 걸린다.

$$\therefore O(m \log n)$$

Alternative Explanations from BFS

모든 Edge Weight가 같을 경우 BFS를 통해 S.P.를 구할 수 있다는 것을 이용한 것이다. 따라서, 노드 사이의 Edge Weight 만큼 더미 노드를 삽입하면 BFS를 통해 S.P.를 구할수도 있다. ($\because$ 모든 Edge Weight가 1로 맞춰짐) 이 경우에는 Priority Queue를 사용하지 않고 그냥 Queue만 쓴다. 하지만 엄청 느려지는 단점이 존재한다.

Dijkstra Selects Node from Nearest to Furthest

Proof. Dijkstra Algorithm 이 집합 R 에 넣는 노드들을 순서대로 $V_1, V_2, \dots, V_n$ 라고 할 때, 모든 k 에 대하여 $d_{min}(V_k) \leq d_{min}(V_{k+1})$ 임을 보이자. (가까운 순서대로 선택하는 것을 보이자.)

V_k 가 집합 R 에 있고, V_{k+1} 이 집합 B 에 있을 때는 정의에 의해 $d_{min}(V_k) \leq d_{min}(V_{k+1})$ 이다. 집합 R 에 V_{k+1} 이 들어간 후, 두 가지 케이스를 고려할 수 있다.

- $d_{min}(V_{k+1})$ 가 바뀌지 않는 경우: 성립한다.
- $d_{min}(V_{k+1})$ 가 바뀌는 경우: $d_{min}(V_{k+1}) = d_{min}(V_k) + g(V_k, V_{k+1}) \geq d_{min}(V_k)$ 이므로 성립한다.

따라서 모든 k 에 대하여 $d_{min}(V_k) \leq d_{min}(V_{k+1})$ 이다. 즉, 이미 집합 R 에 들어간 것들과 direct하게 연결된 edge들만 보면 V_{k+1} 을 반드시 찾을 수 있다. $\square$

3.2 기출문제 Set 3

기출문제 3-1

모든 Weight가 양수인 무향 그래프에서 어떤 두 노드 간의 Shortest Path가 있다면, 그 Path의 어떤 연속된 부분도 그 자체로 Shortest Path임을 증명하라. (12중간) / 그래프의 엣지의 Weight가 모두 양수일 때, Shortest Path의 어떤 일부분도 그 자체로 Shortest Path가 됨을 증명하라. (엣지의 Weight가 모두 양수라는 사실이 증명에 반드시 사용되어야 함에 주의하라.) (13중간)

(증명 보러가기)

기출문제 3-2

Shortest Path 문제를 푸는 Dijkstra 알고리즘에서 노드를 추가하는 순서와 Shortest Path의 길이와의 관계를 정확히 제시하고 증명하라. (13중간) / Shortest Path 문제를 푸는 Dijkstra 알고리즘에서 i 번째로 추가되는 노드 v_i 와 $i+1$ 번째로 추가되는 노드 v_{i+1} 까지의 거리를 비교하면 v_{i+1} 까지의 거리가 v_i 까지의 거리보다 크거나 같음을 증명하라. 모든 엣지의 Weight는 0보다 크다고 가정한다. (14중간)

알고리즘이 추가한 노드들을 순서대로 $V_1, V_2, \dots, V_n$ 라고 하자. 어떤 시점에 V_k 가 최단경로가 구해진 노드 집합 R 에 있고, V_{k+1} 은 아직 최단경로가 구해지지 않은 노드 집합 B 에 있다고 가정하자. 이때는 정의에 의하여 $d_{min}(V_k) \leq d_{min}(V_{k+1})$ 를 만족한다. 이제 집합 R 에 V_{k+1} 이 들어간 후, 두 가지 케이스를 고려하면 된다.

- $d_{min}(V_{k+1})$ 가 바뀌지 않는 경우: 성립한다.
- $d_{min}(V_{k+1})$ 가 바뀌는 경우: $d_{min}(V_{k+1}) = d_{min}(V_k) + g(V_k, V_{k+1}) \geq d_{min}(V_k)$ 이므로 성립한다.

따라서 모든 k 에 대하여 $d_{min}(V_k) \leq d_{min}(V_{k+1})$ 이므로, Dijkstra 알고리즘은 Shortest Path의 길이가 짧은 것 순서대로 노드를 추가한다.

4 Deadline Scheduling, Tape Storage

4.1 Deadline Scheduling

Problem Definition

N 개의 Job $J_1, J_2, \dots, J_N$ 이 있다. 각 Job은 $J_i = (D_i, P_i)$ 으로 구성되어 있고, D 는 Deadline, P 는 Profit을 의미한다. 1시간 단위의 타임 슬롯이 존재하고, 모든 일은 1시간이 소요된다고 가정하자. 이때 가장 Profit을 많이 얻을 수 있는 스케줄링 방법은 무엇일까?

Assumptions and Intuition

- 모든 데드라인은 N 이하라고 가정. ($\because N$ 보다 크면 N 으로 바꿔 생각해도 아무 문제가 없음)
- Job들은 Profit이 큰 것부터 작은 순으로 내림차순 정렬되어 있는 상태라 가정.
- 최적 스케줄에선 데드라인 이후의 일은 안 나온다고 가정.

직관적으로, 높은 이익의 일들을 먼저 빈자리들 중 가장 뒤에 배치하는 것이 제일 효율적이라고 생각할 수 있다.

Proof of Invariant

Invariant: i 번째 Job J_i 를 처리한 직후, 아래 주장을 만족하는 적어도 하나의 optimal solution S 가 존재한다.

스케줄링 알고리즘의 solution을 A 라 하자. i 보다 작은 j 에 대하여, J_i 가 A 에 나오면 J_i 는 S 에도 나오며, 위치가 모두 동일하게 나온다.

Proof. Invariant가 성립함을 보이자.

1. Base: $i = 0$ 일 때: Vacuously True
2. Step: 알고리즘 A 가 J_i 까지 처리한 시점에서 Invariant가 True라고 가정하자.
 - Case 1: A 가 J_{i+1} 을 버리는 경우: A 에서 D_{i+1} 이전의 스케줄이 가득 찼고, 가정에 의해 S 에서도 D_{i+1} 이전도 $J_1, \dots, J_n$ 들 중 일부들로 꽉 찼다는 뜻이다. 그러므로 J_{i+1} 은 S 에도 없다.
 - Case 2: A 가 J_{i+1} 을 slot K ($K \leq N$) 에 배치하는 경우
 - Case 2-1: S 의 slot K 에 J_{i+1} 이 존재하는 경우: 위치가 동일하므로 Invariant가 성립한다.
 - Case 2-2: S 의 slot K' ($K' \leq K$)에 J_{i+1} 이 존재하는 경우: A 에서 slot $K + 1$ 부터 D_{i+1} 까지가 $J_1, \dots, J_n$ 들 중 일부로 가득 찼고, 가정에 의해 S 도 동일하다. S 에서 slot K 와 slot K' 을 교환한 새로운 optimal solution S' 을 생각하자. A 와 S' 이 모두 위치가 동일하므로 Invariant가 성립한다.
 - Case 2-3: S 에 J_{i+1} 이 없는 경우:

- * S 의 slot K 가 비어있는 경우: S 가 정답이 아니므로 모순이 발생한다.
- * S 의 slot K 에 J_x ($x > i + 1$)가 있는 경우: $P_x \leq P_{i+1}$ 이므로 해당 slot에 P_{i+1} 로 바뀌어도 된다.

□

Performance of Deadline Scheduling

균형 트리를 이용하여 $O(N \log N)$ 시간만에 구할 수 있다.

4.2 Tape Storage

Problem Definition

N 개의 Data가 있다. 각 Data는 $D_i = (L_i, F_i)$ 으로 구성되어 있고, L 은 Length of Data, F 는 Frequency of Usage를 의미한다. 테이프에는 각 Data들을 단 한번만 쓸 수 있고, 읽는 것은 무제한으로 가능하다. 하지만 모든 읽기 작업은 항상 테이프의 첫부분부터 읽는다. 이때 가장 효율적으로 데이터를 배치하는 방법은 무엇일까?

Intuition

직관적으로 L_i 가 짧을수록 앞에 배치하고, F_i 가 클수록 앞에 배치하는 것이 좋다고 생각할 수 있다. 따라서 F_i/L_i 의 값이 큰 것부터 내림차순으로 정렬하여 테이프에 쓰면 된다.

Proof of Correctness

증명에 앞서, $C_i(cost) = f_i \sum_{k=1}^n l_k$ 라 하고, 모든 데이터의 cost 총 합이 작을수록 효율적이라고 가정하자.

Proof. F_i/L_i 의 값이 큰 것부터 내림차순으로 정렬하여 테이프에 써야 Cost가 최소이다.

귀류법을 사용하여 증명하자. $F_i/L_i < F_{i+1}/L_{i+1}$ 이라고 가정하자. 먼저 데이터가 1, 2, ..., i , $i+1$, ... 순서로 쓰여져있을 때의 cost를 $Cost_1$ 이라 하자. 그리고 i 와 $i+1$ 번째 데이터의 순서를 바꿔서 1, 2, ..., $i+1$, i , ... 순서로 쓰여져있을 때의 cost를 $Cost_2$ 라고 하자.

$$Cost_1 = f_1 l_1 + f_2(l_1 + l_2) + \dots + f_i(l_1 + \dots + l_i) + f_{i+1}(l_1 + \dots + l_i + l_{i+1} + \dots$$

$$Cost_2 = f_1 l_1 + f_2(l_1 + l_2) + \dots + f_{i+1}(l_1 + \dots + l_{i-1} + l_{i+1}) + f_i(l_1 + \dots + l_{i-1} + l_{i+1} + l_i) + \dots$$

$$Cost_2 - Cost_1 = f_i l_{i+1} - f_{i+1} l_i = l_i l_{i+1} \left(\frac{f_i}{l_i} - \frac{f_{i+1}}{l_{i+1}} \right) < 0$$

$$\therefore Cost_2 < Cost_1$$

Cost가 더 작아지는 솔루션 $Cost_2$ 이 존재하게 되므로, 가정이 모순이다. 따라서 증명 끝!

□

4.3 기출문제 Set 4

기출문제 4-1

Deadline Scheduling 알고리즘의 정확성 증명은 수학적 귀납법 사용하고 있다. 그 증명에서 step 부분은 여러 개의 경우로 나뉘어 있는데, 그 경우들 중 “알고리즘이 $i+1$ 번째 job을 schedule하고, 현재 정답인 T 에도 $i+1$ 번째 job이 등장하는 경우”에 대한 증명을 정확히 제시하라. (만약 수업 시간의 증명이 기억이 나지 않는다면 그 알고리즘에 대한 다른 증명 전체를 제시해도 된다.) (15중간)

알고리즘이 $i+1$ 번째 job을 K 번째 위치에 배치했다고 가정하자.

- T 의 K 번째 위치에 J_{i+1} 이 존재하는 경우: 위치가 동일하므로 Invariant가 성립한다.
- T 의 K' ($K' \leq K$)번째 위치에 J_{i+1} 이 존재하는 경우: 알고리즘에서 $K+1$ 부터 D_{i+1} 까지가 $J_1, \dots, J_n$ 들 중 일부로 가득 찼고, 가정에 의해 T 도 동일하다. T 에서 K 번째와 K' 번째를 교환한 새로운 정답 T' 을 생각하자. 알고리즘과 T' 이 모두 위치가 동일하므로 Invariant가 성립한다.

5 Merge Sort

5.1 Merge Sort Algorithm

Merge Sort는 주어진 배열을 가운데에서 쪼개어 두 개로 만든 뒤 이들을 각각 재귀 호출을 하여 정렬한다. 그 후 정렬된 두 배열을 Merge 알고리즘을 통하여 하나로 합쳐서 정렬된 배열을 얻는다.

5.2 Merge Algorithm

배열 A, B를 병합하여 배열 C에 저장하고자 한다. A, B의 원소들을 가리키는 포인터가 각각 존재한다고 하면 처음에는 그 포인터가 $A[0], B[0]$ 에 존재한다. 배열 C에는 두 포인터가 가리키는 값 중 더 작은 값이 들어가게 되고, 그 값을 가리키던 포인터는 한칸 뒤로 이동하게 된다. 이 과정을 두 포인터가 모두 배열의 끝에 갈때까지 반복한다. 이 알고리즘 자체는 포인터가 이동하면서 n 개의 원소를 이동시키므로 $O(n)$ 이 걸린다.

5.3 Merge Sort의 정확성 증명

1. 집합 조건을 깰 수 있는 코드는 없다.
2. $n = 1$ 인 경우: 할 일이 없다.
3. $n/2$ 일 때 `sort()` 함수가 성공한다고 가정하자. (`sort()` 함수가 실패할 경우는 할 말이 없다.) 즉, 재귀 호출이 끝난 후 새로 생성된 배열 c 에 대하여 $c[0] < c[1] < \dots < c[n/2 - 1]$ 이고, $c[n/2] < c[n/2 + 1] < \dots < c[n - 1]$ 이다.
4. n 일때는 merge 알고리즘의 정확성 (집합관계가 깨지지 않고, sorted 된 상태로 merge 됨)에 의해 `sort()` 함수가 성공한다.
5. 시간복잡도: $T(n) = n + 2T(n/2) = n + 2(n/2 + 2(T(n/4))) = \dots = O(n \log n)$

5.4 기출문제 Set 5

기출문제 5-1

merge sort의 알고리즘을 제시하라. (12중간, 17중간) / Merge 알고리즘을 제시하라. (14중간)

기출문제 5-2

정렬되어있는 두 자연수 배열을 merge해 $2n$ 짜리 배열을 만들었다. k 번째에 올 값을 구할수 있는가? 구할수 있다면 알고리즘을 제시하고, 알고리즘의 속도를 구하라. (17중간)

(기출문제 6-2와 동치) Stack overflow Source Code

6 Quick Sort

6.1 Quick Sort의 Idea

Merge Sort는 O notation 상으로는 가장 빠르지만 배열 할당할 때 메모리 소비가 커서 n 이 커지면 잘 안쓰게 된다. Quick Sort는 추가 메모리를 할당하지 않는 것이 목적이다.

1. 배열에서 어떤 값 하나를 pivot으로 설정한다.
2. 배열의 가장 앞과 가장 뒤 값을 가리키는 포인터를 각각 i, j 라 하자. i 와 j 가 겹칠때까지 아래의 과정을 반복한다.
3. i 가 가리키는 값이 피벗보다 작으면 i 를 뒤로 옮겨서 피벗보다 커질때까지 옮기고, j 가 가리키는 값이 피벗보다 크면 j 를 앞으로 옮겨서 피벗보다 작아질때까지 옮긴다. 둘 다 멈춘 뒤 i 가 j 보다 작으면 i 가 가리키는 값과 j 가 가리키는 값을 교환한다.
4. pivot과 j 가 가리키는 값을 교환한다.
5. 이제 처음부터 j 가 가리키는 곳까지 / j 가 가리키는 곳 다음부터 배열의 끝까지를 각각 재귀적으로 정렬한다.

```

1 int qsort(int a[], int n) {
2     // if n <= 5 do selection sort
3     int p = a[0];
4     int i = 1; int j = n-1;
5     while (i < j) {
6         while (a[i] < p) i++;
7         while (a[j] > p) j--;
8         if (i < j) swap(a[i], a[j]);
9     }
10    swap(a[0], a[j]);
11    qsort(a, j); qsort(a+j+1, n-j-1);
12    return;
13 }
```

6.2 Quick Sort의 정확성 증명

1. 집합 조건을 깰 수 있는 코드는 없다.
2. $n \leq 1$ 인 경우: 할 일이 없다.
3. 함수 내의 재귀함수가 성공한다고 가정하자. 즉, 재귀 호출이 끝난 후 $a[0] < a[1] < \dots < a[j-1]$ 이고, $a[j+1] < a[j+2] < \dots < a[n-1]$ 이다.
4. 재귀 호출이 일어나기 전에 $a[j-1] < p == a[j] < a[j+1]$ 이 성립했다. 따라서, 함수가 끝났을 때 $a[0] < a[1] < \dots < a[n-1]$ 이 성립하므로, qsort() 함수가 성공한다.

6.3 Quick Sort 시간 복잡도

Pivot이 뭐냐에 따라서 성능이 달라짐! 대부분 $O(n \log n)$ 으로 작동함.

- Worst Case : $T(n) = T(n-1) + n = \dots = O(n^2)$
- Best Case (피벗이 중간일때): $T(n) = 2T(n/2) + n = \dots = O(n \log n)$
- Average Case (배열의 모든 값이 피벗으로 선택될 확률이 동일하다고 가정): $T(n) = O(n \log n)$

성능 향상 시키는 Idea: 애초에 피벗을 중간값으로 잘 뽑으면 Worst Case의 시간 복잡도가 더 향상될 것이다. Pivot Selection 중 Approximate Median이 $O(n \log n)$ 으로 풀려서 Quick Sort도 $O(n \log n)$ 에 풀리게 된다.

6.4 기출문제 Set 6

기출문제 6-1

Quick Sort 알고리즘을 제시하고 정확성을 증명하라. (12중간) / Quick Sort의 첫 부분 (배열을 두 개로 나누는 부분)의 알고리즘을 제시하라. (증명은 필요 없음.) (13중간)

6.1절, 6.2절 참고

기출문제 6-2

값이 커지는 순서로 정렬된 배열 $A[1], A[2], \dots, A[n]$ 에서 $A[i]=i$ 가 되는 경우가 있으면 찾아내는 Divide-and-Conquer 방식의 알고리즘을 제시하고 세 줄 이내로 간략히 증명하시오. 단, 배열의 모든 값은 다르다고 가정한다. (13중간)

Stack overflow Source Code

7 Approximate Median Problem

7.1 Selection Strategy with Approximate Median

$a_1, a_2, \dots, a_n$ 중에서 $[rn, (1-r)n]$ 등까지 있는 원소를 찾는다. 예를 들어 $r = 0.3, n = 100$ 인 경우 $[30, 70]$ 등 사이에 있는 것들을 Approximate Median 이라고 한다.

Selection 문제는 n 개의 원소에서 k 등(정렬했을때 k 번째) 원소를 찾는 것이다. 이때도 pivot을 이용하면 $O(n)$ 의 시간만에 구할 수 있다. Quick Sort에서 pivot을 기준으로 왼쪽 오른쪽으로 분류한 것과 같이 나눈 뒤 k 등이 pivot 보다 왼쪽에 있으면 왼쪽 부분만 재귀적으로 Selection을 수행한다. k 등이 pivot보다 오른쪽이면 오른쪽 부분만 재귀적으로 Selection을 수행한다. 이 과정을 반복하면 $O(n)$ 시간만에 k 등 원소를 찾을 수 있다.

1. pivot이 중간값 (Median)인 경우: $T(n) = T(n/2) + n = \dots = O(n)$
2. pivot이 Approximate Median($r = 0.3$) 인 경우: $T(n) = T(0.7n) + n = T(0.49n) + 0.7n + n = \dots = O(n)$

7.2 Approximate Median과 Quick Sort

Quick Sort에서 $r = 0.3$ 일때 Approximate Median 으로 피벗을 구해도 $O(n \log n)$ 의 시간이 걸린다. Worst Case에서 피벗은 Approximate Median의 가장 양 끝값인 경우가 될 것이다. $\therefore T(n) = T(0.3n) + T(0.7n) + n = O(n \log n)$

Quick Sort에서 피벗을 선택하기 위하여 Selection 문제를 먼저 풀어야 한다. Selection 문제를 풀기 위하여 Approximate Median 알고리즘을 이용한다. 즉 pivot을 선택하기 위해 A.M 알고리즘을 돌리면 $O(n)$ 만에 선택할 수 있고, 이 경우 Quick Sort도 Worst Case에도 $O(n \log n)$ 만에 풀 수 있다. (하지만 이 경우 배열을 추가로 할당해야하는 문제점이 존재한다. 그래서 잘 쓰진 않음)

7.3 Approximate Median Algorithm

$r = 0.3$ 일때 Approximate Median 을 찾는 알고리즘을 알아보자. 원소들을 5개씩 잘라서 하나의 그룹으로 만들고 그 그룹 내에서 정렬을 하면 $O(n)$ 시간안에 정렬이 된다. (아래에서 위 방향으로 커지게 정렬) 이제 각 그룹 내에서 중간값 (3등) 들끼리 모아서 Sorting 되었다고 가정하자. (실제로 정렬하진 않음, 나머지는 따라 움직인다고 생각) 3등들끼리 모은 배열에서 또 중간값을 Median X(초록색)라고 하자. 이 경우 배열 전체에서 $0.3n$ 개 (빨간색 그룹)는 항상 X보다 작고, $0.3n$ 개(파란색 그룹)는 항상 X보다 큼을 알 수 있다. 따라서 X는 Approximate Median이 된다.

○	○	○		○		○	○	○
○	○	○		○		○	○	○
○	○	○	...	○	...	○	○	○
○	○	○		○		○	○	○
○	○	○		○		○	○	○

7.4 Approximate Median Analysis

n 개에서 A.M.을 푸려면 5개끼리 묶어서 그룹화를 하고($O(n)$), $n/5$ 개의 중간값들 중 Median을 찾아야 했다. 여기서 $n/5$ 개짜리 Selection을 해야 한다. (Selection을 하기 위해 A.M.을 쓰는데 그 안에 $n/5$ 개짜리 Selection이 있는 셈)

S 를 Selection을 하는 시간, A 를 A.M.을 하는 시간이라고 하자. $S(n)$ 은 n 개짜리 Approximate Median 알고리즘을 돌림: $A(n)$ + 피벗 찾아서 왼쪽 오른쪽으로 구분: $O(n)$ + Selection을 다시 호출 (최대 Size: $S(0.7n)$)

A 안으로 또 들어가면 피벗 왼쪽 오른쪽으로 구분하는 $O(n)$ + $n/5$ 개짜리 Selection 시간 $S(0.2n)$

$$\therefore S(n) = S(0.2n) + S(0.7n) + n, \quad S(n) = O(n)$$

여기서 0.2와 0.7의 합이 1보다 작기 때문에 $O(n)$ 이 걸렸다. 만약 1이면 $O(n \log n)$ 의 시간이 걸렸을 것이다. 그래서 아까 $1/5$ 씩으로 끊은 것이다!

7.5 기출문제 Set 7

기출문제 7-1

$r=0.3$ 으로 둔 경우의 Approximate Median 문제를 해결하는 알고리즘을 제시하고 정확성을 증명하라. 단, 시간 복잡도는 계산할 필요가 없다. (14중간)

기출문제 7-2

Approximate Median을 이용하여 Selection을 수행하는 알고리즘의 시간 복잡도를 분석하라. (14기말, 17기말)

기출문제 7-3

자연수가 정렬되어 저장된 두 개의 크기 N 인 배열이 있다. 이 두 배열의 자연수들 중 중간값을 찾는 알고리즘을 제시하고 정확성 및 시간 복잡도를 증명하시오. 두 배열에 있는 값들은 모두 다르다고 가정해도 좋다. $O(N)$ 보다 빠른 알고리즘만 점수를 받을 수 있다. (15중간)

8 Closest Pair

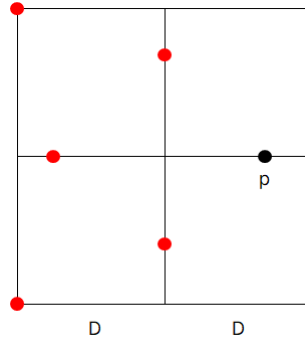
8.1 Closest Pair Idea

주어진 점들 중에서 가장 가까운 쌍을 찾는 문제 유형이다. 1차원인 경우에는 단순히 정렬을 통해서 구할 수 있고, 2차원의 경우 두 점 사이의 거리 공식을 이용해 일일이 비교하여 구할 수 있다. 하지만 이 경우 시간 복잡도가 $O(n^2)$ 이나 걸리므로 분할 정복을 이용하여 효율성을 높일 수 있다.

1. 점들을 x좌표에 따라 오름차순으로 정렬한다.
2. x좌표를 기준으로 배열을 반반으로 분할한다. (*basecase* : $n \leq (2 \text{ 이상의 자연수})$)
3. 왼쪽에서의 답과 오른쪽에서의 답을 각각 DL, DR 이라 하자. 그리고 $D = \min(DL, DR)$ 을 구하면서 재귀적으로 합치면 된다.

하지만 왼쪽 영역의 점과 오른쪽 영역의 점이 합쳐진 경우가 존재하여 D 가 답이 아닐 가능성이 있다. 그렇다고 왼쪽의 모든 점과 오른쪽의 모든 점끼리 비교할 수는 없다. 이를 해결하기 위해서 밴드라는 아이디어를 생각해보자. 가상의 수직선을 긋고, 왼쪽으로 D , 오른쪽으로 D 만큼씩 이동하여 경계를 만들어서 밴드를 생성한다. 이 밴드 내에서만 왼쪽 점 하나, 오른쪽 점 하나를 비교하면 D 보다 작은 경우를 쉽게 찾을 수 있다.

8.2 $O(n \log^2 n)$ 풀이



1. 밴드의 내부에 한 변의 길이가 $2D$ 인 정사각형을 생각하자. 여기서 찾고자 하는 것은 어떤 점 p 에 대해서 D 보다 가까운 점이 있는지이다. p 는 정사각형 오른쪽 영역의 가운데 높이에 위치한다고 가정하자.
2. 밴드 내의 점들끼리 거리는 D 보다 커야 되므로 p 와 비교해야할 점은 최대 5개이다.
3. 이를 일반화하면 각 밴드 내의 점들이 모두 5개씩 비교를 해서 D 보다 작은 거리가 있는지를 찾아야 한다. 밴드 안에 있는 모든 점들을 y좌표에 따라 오름차순 정렬을 한 뒤 왼쪽 오른쪽 영역 구분 없이 본인보다 위에 있는 5개의 점을 선택해서 비교하면 된다. 여기서 D 보다 짧은 거리가 있으면 그게 답이고, 없으면 D 가 답이다.

8.3 $O(n \log n)$ 풀이

밴드를 만든 후, 밴드 안에 있는 것만 보는 것이 아니라 모든 점들을 y 좌표에 따라 오름차순 정렬을 한 뒤 5개의 점을 선택해서 비교한다. 이렇게 하면 밴드 내에서 D 보다 작은 거리가 있는지 찾는 작업은 똑같이 $O(n)$ 만 걸린다. 왼쪽 영역에서 y 좌표 정렬, 오른쪽 영역에서 y 좌표 정렬된 것끼리 다시 합칠 때는 Merge를 하므로 이 때 $O(\log n)$ 만 걸리게 되어 총 시간은 $O(n \log n)$ 이 걸린다. 3번 풀이에서는 정렬하는 시간이 $O(n \log n)$ 이 소요되었지만 여기서는 Merge를 통해 시간을 $\log n$ 만큼 줄여서 최종적으로 $O(n \log n)$ 의 시간만에 Closest Pair를 구할 수 있다.

8.4 Plane Sweeping 풀이

점들을 x 좌표 오름차순으로 정렬하고, 수직선이 점들을 지나가면서 지금까지 본 것들 중에서 가장 가까운 거리를 수직선에 저장한다. (수직선에는 밴드의 점들이 y 좌표 순으로 정렬되어 AVL 트리 / Red black 트리 형태로 저장된다.)

8.5 기출문제 Set 8

기출문제 8-1

Closest Pair 문제를 푸는 알고리즘은 $O(n \log n)$ 시간 이상을 사용해야 함을 증명하라. (15 중간)

각 점들을 x 좌표 오름차순으로 정렬하고 기준선 l 을 기준으로 영역을 반으로 분할한 뒤 재귀적으로 왼쪽과 오른쪽 영역에서의 답을 각각 구하였을 때 둘 중에서 작은 값을 D 라고 하자. l 을 기준으로 좌우로 각각 거리가 D 인 밴드를 만들고, 밴드 안에 있는 모든 점들을 y 좌표 오름차순으로 정렬한 뒤 각 점마다 5개씩 거리를 비교하여 정답을 갱신하면 $O(n(\log^2 n))$ 의 시간이 걸린다. 밴드에 있든 없든 모든 점들을 y 좌표 오름차순으로 정렬한 뒤 밴드 안에 있는 점들끼리 각 점마다 5개씩 거리를 비교하여 정답을 갱신하면, 기준선의 왼쪽 영역과 오른쪽 영역에서 점들을 합칠 때는 각각 서로 y 좌표로 정렬된 점들을 Sort하지 않고 Merge만 하여 합칠 수 있다. 따라서 이 경우에는 정렬하지 않고 Merge만 하므로 시간이 $O(n \log n)$ 만 걸리게 된다.

기출문제 8-2

평면상의 가장 가까운 점의 쌍 찾기 문제를 해결하는 Plane Sweep 스타일의 알고리즘을 제시하고 그 알고리즘의 시간 복잡도에 대해 간략히 설명하라. (14중간)

기출문제 8-3

좌표평면에 짝수 개의 점이 주어져 있고, x 좌표나 y 좌표가 같은 점의 쌍은 없고 세 점이 동일 직선 위에 있는 경우도 없다고 하자.

(a) 모든 점을 다른 하나의 점과 선분으로 연결하여 선분들이 교차하지 않도록 만들 수 있음을 증명하라. (10점)

(b) 실제로 그러한 점들의 짝 맞추기를 찾을 수 있는 알고리즘을 제시하라. (10점)

참고로, x 좌표가 모두 다르다는 것만 이용하여 매우 간단한 알고리즘을 만들 수 있으며, 이 알고리즘이 정확하다는 것을 증명하는 것으로 (a)부분의 답도 제시할 수 있다. 다만 이 경우 증명은 세세하고 정확해야 한다. (a) 부분의 증명을 독립적으로 제시하는 경우에는 알고리즘의 정확성에 대한 것은 직관적 수준의 설명으로 충분하다. (14기말)

9 Convex Hull

9.1 Convex Hull, CCW Algorithm

Convex Hull 은 평면좌표에서 어떤 점들이 있을 때 모든 점들을 포함하는 가장 작은 볼록 다각형을 의미한다. (점들의 바깥에서 고무줄을 뺐을 때 만들어지는 도형) 점들을 정렬하는 과정이 필요하기 때문에 Convex Hull 을 만드는 알고리즘의 시간 복잡도는 $O(n \log n)$ 보다 빠를 수 없다.

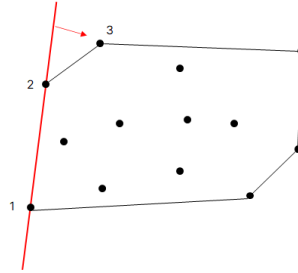
앞으로 세 점의 관계를 알아낼 때 CCW (Counter Clock Wise) 알고리즘을 사용하는 경우가 많으므로 미리 알아두자. 세 점 $(x_1, y_1), (x_2, y_2), (x_3, y_3)$ 에 대해서 $1 \rightarrow 2 \rightarrow 3$ 방향으로 이동할 때 아래 값을 이용하여 회전 방향을 판단할 수 있다.

$$x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2)$$

위의 값이 양수면 좌회전 (반시계), 음수면 우회전 (시계), 0이면 일직선 상의 세 점을 의미한다.

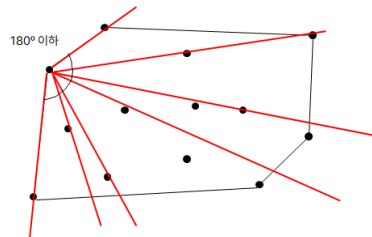
9.2 Brute-Force-ish (1) - $O(n^3)$

모든 선분마다 Convex Hull 의 일부인지를 판단하는 알고리즘이다. 선분에 포함된 두 점과 다른 모든 점들을 각각 이어봤을 때, 모두 같은 방향으로 회전하면 Convex Hull의 일부이다.



9.3 Brute-Force-ish (2) - $O(n^2 \log n)$

특정 점이 Convex Hull 에 포함되는 꼭짓점인지를 판단하는 알고리즘이다. 한 점에 대해서 모든 점들로 선분을 이었을 때, 선분이 존재하는 범위의 각도가 180도 이하 (외부 각도 기준으로 180도 이상) 이면 Convex Hull 에 포함되는 점이다. 이 경우 Sorting이 필요하므로 $O(n \log n)$ 이 소요된다.



9.4 Package Wrapping - $O(n^2)$

Convex Hull 위에 있는 것이 보장된 점 (ex. y좌표가 가장 작은 점) 위에서 시작한다. 각 점마다 모든 점들 중 가장 왼쪽 / 오른쪽에 있는 점을 찾아서 연결하고, 그 점에서 다시 반복하여 연결하면 된다. 이 경우 최댓값을 찾는 알고리즘 처럼 Sorting 할 필요 없이 바로 비교를 시작하여 매 쌍마다 오른쪽에 있는 점을 고르면 된다. 따라서 $O(n^2)$ 만큼 시간이 걸린다.

9.5 Graham Scan

Convex Hull 을 찾는 가장 유명한 알고리즘이다. 기본적인 원리는 각도순으로 점을 보며 스택에 넣다가, 다음 점을 연결할 때 우회전할 경우 다시 좌회전이 되도록 지금까지 추가한 점들을 스택에서 제거하는 것이다. 알고리즘 자체는 모든 점들이 스택에 push/pop만 수행하므로 $O(1)$ 밖에 안걸리고, 점들을 정렬하는데는 $O(n \log n)$ 시간이 걸린다.

1. 2차원 평면의 점들을 정렬하고 가장 아래에 있는 점 P0를 고르고 스택에 넣습니다.(여러 개일 경우 가장 왼쪽에 있는 점)
2. 해당 점과 나머지 점들의 기울기를 구해 x축과의 각도를 구합니다.
3. 해당 각도를 기준으로 오름차순 정렬합니다.
4. P0와 각도가 가장 작은 P1과 그 다음으로 작은 P2를 고릅니다. P0, P1, P2에 대해서 CCW 알고리즘을 사용하고 만약에 양수 값(반시계방향)이면 스택에 P1, P2를 넣습니다.
5. 다음 점을 P3로 놓고, P1, P2, P3에 대해서 CCW 알고리즘을 사용합니다. 만약 0이거나 음수 값이라면 스택을 pop하고, P3를 스택에 넣어줍니다.
6. 다음 점 P4를 놓고, P1, P3, P4에 대해서 CCW 알고리즘을 수행합니다.

9.6 Dynamic Case - Plane Sweeping

Convex Hull 이 이미 만들어졌는데 점을 하나 추가하거나 제거하는 Dynamic Case 에서는 Plane Sweeping 을 사용한다. Closest Pair 에서 사용한 Balance Tree 수직선을 이용하여 점이 추가/제거 되는 경우를 쉽게 해결할 수 있다. - 접선을 찾아서 업데이트하는 방식

9.7 Divide and Conquer - $O(n \log n)$

Upper hull 기준으로 적용해보자. 기본적인 아이디어는 Left Hull과 Right hull 의 공통접선을 찾아서 연결하는 것이다. 공통접선을 찾는 과정에서 왼쪽은 $\log n$ 개의 점을 탐색하고, 오른쪽에서는 점 하나당 다시 $\log n$ 개의 점들을 탐색하므로 총 시간은 $\log^2 n$ 만큼 걸리게 된다.

10 Dynamic Programming

10.1 Select Working Days

문제 정의: N일 동안 각 날짜마다 Pay가 주어진다. 연속으로 일을 할 수 없을 때, N일간 Total Pay를 최대화할 수 있는 방법을 구해보자.

Idea: k ($k \leq n$)일까지 벌 수 있는 가장 큰 금액을 k 일에 일을 했을 경우와 안 했을 경우로 나눠서 구하면 된다.

- k 일에 일을 안한 경우: $\max(k-1$ 일에 일한 경우, $k-1$ 일에 일을 안한 경우)
- k 일에 일을 한 경우: $k-1$ 일에 일을 안한 경우 + k 일 Pay

```
1 int D[n+1][2];
2 D[1][0] = 0; D[1][1] = a[1];
3
4 for (i=2; i<=n; i++){
5     D[i][0] = max(D[i-1][0], D[i-1][1]);
6     D[i][1] = D[i-1][0] + a[i];
7 }
```

10.2 Matrix Chain Multiplication

문제 정의: 행렬들을 곱하는 순서에 따라 걸리는 시간이 달라진다. 행렬 $M_1, M_2, \dots, M_n$ 들을 곱한 결과를 구하는 데 걸리는 시간을 줄이는 것이 목표이고, 각 행렬들의 크기는 $M_k = d_{k-1} \times d_k$ 이다. 즉, 행렬 M_i 번 부터 M_j 번 까지 곱하는데 걸리는 비용을 C_{ij} 라 하면, 목표는 C_{1N} 을 구하는 것이다.

Idea: $j-i$ 의 값이 작은 것부터 C를 계산하면 된다. $j-i$ 가 1일 때는 비용이 0이다. $j-i$ 가 2일 때는 두 행렬의 곱이므로 $C_{i \ i+1} = d_{i-1}d_i d_{i+1}$ 이다. $j-i$ 가 3 이상일 때는 행렬 그룹을 2개로 나눴을 때 가장 비용이 작은 그룹끼리 곱한 것을 택하면 된다. (각 행렬 그룹 내의 행렬들의 곱은 $j-i$ 가 더 작았을 때 구해놓은 상태)

```
1 int D[n + 1][n + 1]; // D[i][j] = C(i->j)
2
3 for (i = 1; i <= n; i++)
4     D[i][i] = 0;
5 for (s=1; s<=n-1; s++)
6     for (i = 1; i <= n - s; i++) {
7         minval = INF;
8         for (k = i; k <= i + s - 1; k++)
9             minval = min(minval, D[i][k] + D[k + 1][i + s] + d[i - 1] * d[k] * d[i + s]);
10        D[i][i + s] = minval;
11    }
```

10.3 Maximum SubArray

문제 정의: 연속적인 Subarray 중 합이 최대가 되는 Subarray의 합을 구해보자. (아래 풀이에서 Step에서는 기본적으로 크기가 0인 배열 (공집합 배열)은 허용하지 않는다고 가정한다.)

Idea 1: 인덱스 i 번에서 끝나는 Maximum Subarray를 계산한다.

- Base: $i = 0$ 이면 첫번째 항이 답
- Step: i 일 때 답이 $S[i]$ 라고 하자. $S[i+1] = \max(S[i] + A[i+1], A[i+1])$ (단, 공집합 배열을 허용하면 $S[i+1] = \max(S[i] + A[i+1], 0)$)

Idea 2: 인덱스 i 번 차례에서 자신을 포함하는 경우와 포함하지 않는 경우로 나뉘어서 계산한다. 즉, i 번 칸을 포함하는 것 중 제일 큰 것과 i 번 칸을 포함하지 않는 왼쪽 전체 중에서 제일 큰 것을 나뉘어서 구하면 된다. i 일 때 답이 $S[i]$ 라고 하자.

- $i+1$ 일 때 자신을 포함하는 경우: $S[i+1] = \max(S[i] + A[i+1], A[i+1])$
- Step: $i+1$ 일 때 자신을 포함하지 않는 경우: $S[i+1] = \max(S[i], i+1$ 번을 포함하는 경우의 답) (단, 자신을 포함하지 않는 경우에서 공집합 배열을 허용하면 $S[i+1] = \max(S[i], 0)$)

Idea 3: Prefix Sum을 이용하여 풀 수 있다. 부분합 P_i 를 A_i 까지 더한 합이라고 하자. 그럼 i 번에서의 정답은 $\max(P_i - P_{i-1}, P_i - P_{i-2}, \dots, P_i - P_0)$ 이다. 즉, $\min(P_{i-1}, P_{i-2}, \dots, P_0)$ 을 찾아서 빼면 된다.

10.4 Floyd-Warshall Algorithm

문제 정의: 모든 노드의 쌍들마다 최단 경로를 구해보자. 쉬운 방법으로는 Dijkstra 알고리즘을 n 번 돌리기가 있다.

Idea: 각 노드별로 번호를 부여했다고 가정하자. $i \rightarrow j$ 로 가는 최단경로를 구할 건데, $1, 2, \dots, k$ 번 노드 중에서만 거쳐가는 최단 경로를 찾으면 된다. 이 과정을 k : 0부터 N 까지 반복한다. 즉, 구하고자 하는 것은 모든 (i, j) 쌍에 대하여 1번부터 n 번 노드 중에서만 거쳐서 갈 수 있는 최단경로이다.

Computing: k 일 때 노드 k 를 쓰는 경우와 노드 k 를 쓰지 않는 경우로 나뉘어서 더 짧은 경로를 구하면 된다.

- 노드 k 를 쓰지 않는 경우: $D[k][i][j] = D[k-1][i][j]$
- 노드 k 를 쓰는 경우: $D[k][i][j] = D[k-1][i][k] + D[k-1][k][j]$ - k 이전까지 경로 + k 이후의 경로로 연결

```

1 int D[n + 1][n + 1][n + 1];
2
3 for (i = 1; i <= n; i++)
4     for (j = 1; j <= n; j++)
5         D[0][i][j] = W[i][j];
6
7 for (k = 1; k <= n; k++)
8     for (i = 1; i <= n; i++)
9         for (j = 1; j <= n; j++)
10            D[k][i][j] = min(D[k-1][i][k], D[k-1][i][k] + D[k-1][k][j]);

```

2차원 배열 Floyd-Warshall

$D[k-1][i][k] == D[k][i][k]$, $D[k-1][k][j] == D[k][k][j]$ 이다. 왜냐하면 출발점이나 도착점이 k 번 정점일 때 k 번을 경유하는 것은 아무 의미가 없기 때문이다. 따라서 아래와 같이 코드를 개선할 수 있다.

```

1 // 2 Array : D[k][i][k] == D[k-1][i][k]
2
3 int D[n + 1][n + 1];
4
5 for (i = 1; i <= n; i++)
6     for (j = 1; j <= n; j++)
7         D[i][j] = W[i][j];
8
9 for (k = 1; k <= n; k++)
10    for (i = 1; i <= n; i++)
11        for (j = 1; j <= n; j++)
12            D[i][j] = min(D[i][k], D[i][k] + D[k][j]);

```

10.5 Edit Distance

문제 정의: 편집 거리는 한 문자열을 다른 문자열로 변경하기 위해 필요한 편집 연산들의 최소 수를 의미한다. 연산의 종류는 [삽입, 삭제, 교체, 유지]가 있다.

Idea: $D(i, j)$ 는 두 문자열 S_1, S_2 에 대한 편집거리를 나타낸다고 하자. 제일 오른쪽 문자부터 비교를 하면서 연산들 중 값이 가장 작아지는 것을 고르면 된다.

$D(i, j)$ 의 값은 아래 4가지 값들 중 가장 작은 값이다.

- $D(i, j - 1) + 1$ - 삽입
- $D(i - 1, j) + 1$ - 삭제
- $D(i - 1, j - 1) + \text{diff}(i, j)$ - 같으면 유지(0), 다르면 교체 (1)

10.6 LCS (Longest Common Subsequence)

문제 정의: 부분 서열 (Subsequence)은 주어진 서열에서 일부 문자를 삭제한 후에 남은 문자들을 순서대로 연결하여 만들어지는 서열을 의미한다. LCS는 X와 Y가 주어질 때 X와 Y의 공통 부분 서열 중 가장 긴 것을 의미한다. X의 길이가 m, Y의 길이가 n일때 X와 Y의 LCS의 길이를 구해보자.

Idea: $LCS(i, j)$ 를 X_i, Y_j 의 LCS의 길이라고 하자. 우리가 구해야할 값은 $LCS(m, n)$ 이다. 기본적인 아이디어는 위의 편집거리와 유사하다.

- $LCS(i, j) = 0$: $i = 0$ 이거나 $j = 0$ 인 경우
- $LCS(i, j) = LCS(i - 1, j - 1) + 1$: X_i, Y_j 가 같으면 LCS의 길이가 1 추가된다.

- $LCS(i, j) = \max(LCS(i-1, j), LCS(i, j-1))$: X_i, Y_j 가 다르면 삽입 또는 삭제 연산을 통해 다음 문자를 보면 된다.

10.7 Global Alignment

문제 정의: 두 개의 문자열을 "-"을 포함하도록 확장하여 같은 길이의 문자열로 만드는데 대응되는 문자들의 쌍들의 score 합이 최대가 되는 alignment를 구해보자.

Idea: $\sigma(x, y)$ 를 문자 x 와 문자 y 에 대한 유사도라 하자. 유사도는 $x = y$ 이면 점수가 많고, $x \neq y$ 일 때 빈칸이 적을수록 점수가 더 많다. $\sigma(-, -)$ 는 0 이하의 점수이다. 이번에도 편집 거리와 유사한 방식으로 두 Sequence A, B의 유사도를 가장 크게 만드는 Global Alignment를 구해보자. $V(i, j)$ 는 A_i, B_j 사이의 유사도이고, 최종 정답은 $V(m, n)$ 이다.

[기저 조건]

- $V(0, 0) = 0$
- $V(i, 0) = V(i-1, 0) + \sigma(A_i, -)$
- $V(0, j) = V(0, j-1) + \sigma(-, b_j)$

$V(i, j)$ 는 아래 세 가지 값들 중 최댓값이다.

- $V(i-1, j-1) + \sigma(A_i, B_j)$
- $V(i-1, j) + \sigma(A_i, -)$
- $V(i, j-1) + \sigma(-, B_j)$

10.8 최대 공백 정사각형

문제 정의: 주어진 $n \times n$ 크기의 흑백 이미지에서 검은 점을 포함하지 않는 가장 큰 빈 정사각형 찾기

Idea: $LES(x, y)$ 를 (x,y)를 우측하단 꼭짓점으로 하는 최대 정사각형의 크기라고 하자.

- $LES(x, y) = 0$: (x,y)가 비어있지 않은 경우
- $LES(x, y) = 1$: 첫번째 행 또는 열의 (x, y)가 비어있는 경우
- $LES(x, y) = \min(LES(x-1, y-1), LES(x, y-1), LES(x-1, y)) + 1$: 나머지 픽셀 (x,y)가 비어있는 경우 (하나라도 작으면 작은 사이즈의 정사각형을 가지고 계산해야함)

10.9 동전 거스름돈

문제 정의: N원의 거스름 돈을 돌려줄 때, 최소 개수의 동전을 사용하여 거스름 돈을 돌려주는 방법을 구해보자.

Idea: $C(i, j)$ 를 크기 D_i 동전으로 j 원을 거슬러 줄때의 동전 개수라고 가정하자. $C(i, j)$ 는 아래 두 값 중 더 작은 값이다.

- $C(i, j - D_i) + 1$: D_i 원을 거슬러준 경우 동전 하나 추가
- $C(i - 1, j)$: 거슬러 주지 않고 다음 돈으로 스킵

10.10 기출문제 Set 10

기출문제 10-1

Floyd의 알고리즘에서 $d_{min}^{k-1}(i, j)$ 와 $d_{min}^{k-1}(i, k) + d_{min}^{k-1}(k, j)$ 중 작은 것을 $d_{min}^k(i, j)$ 의 값으로 치환하는 과정을 반복한다. 그러면 예를 들어 $d_{min}^k(i, k)$ 의 값은 실제로 $d_{min}^{k-1}(i, k)$ 인지 $d_{min}^k(i, k)$ 인지 구분하지 않고 사용하게 되는데 그럼에도 불구하고 알고리즘이 정확히 동작하는 이유를 설명하라. (14기말)

기출문제 10-2

All pair of Shortest Path를 찾는 알고리즘을 제시하라. (17기말)

+LIS 알고리즘

11 Graph Traversal

11.1 Any-Order Traversal

Any-Order Traversal's Reachability

Anyorder Traversal: 어떤 노드 s 에서 시작하고 s 를 BOX에 넣는다. BOX가 비어있지 않으면 아래의 과정을 반복한다. 하나의 노드를 꺼내고 방문하지 않은 노드라면 방문한 뒤 어떤 계산을 수행한다. 그리고 노드와 인접한 모든 엣지들을 BOX에 넣는다. 이후 이 과정을 계속 반복한다.

AnyOrder Traversal이 Reachable한 노드는 모두 방문한다는 것을 증명하자. 노드 t 가 s 에서 reachable 하다고 가정하자. (Box에 들어가는 노드는 모두 reachable 하다는 것은 invariant 이다.)

1. $t = s$ 인 경우 s 를 방문하는 것이 t 를 방문하는 것이므로 성립한다.
2. $s \rightarrow a_1 \rightarrow a_2 \rightarrow \dots \rightarrow t$ 를 통해서 도달한다고 하자. 이 sequence에서 t 보다 앞에 있는 노드 a_i 까지만 방문해서, t 를 방문하지 않는다고 가정하자. (귀류법) 따라서 a_{i+1} 은 방문하지 않은 노드이다. 이때 a_{i+1} 은 a_i 와 인접한 노드이므로 Box에 들어간다. 알고리즘에 의해 a_{i+1} 은 Box에서 꺼내지므로 방문하게 된다. 모순 발생!

이제 알고리즘별로 어떤 BOX를 쓰는지 확인해보자.

- DFS: Stack
- BFS: Queue
- Prim: PQ with edge weight
- Dijkstra: PQ with distance (from s)
- A*: PQ with Arbitrary Function

Connected Component

모든 노드에 대해서 Anyorder Traversal을 하면 찾을 수 있다. CC는 노드의 집합 / 모든 쌍이 연결됨 / 더이상 노드를 추가할 수 없는 것 이다.

11.2 Recursive DFS

RDFS 는 시작노드로부터 인접한 노드로 방문하는데, 앞 노드가 방문되지 않았으면 그 노드에서 또 재귀적으로 호출한다. 각 노드들이 RDFS의 인자가 되는 것은 한번 뿐이므로 시간 복잡도는 $O(m+n)$ 이 걸린다.

Edge의 종류

DFS에서 전진한 edges로만 생성한 결과를 DFS Tree라 한다. 시작 노드가 루트이고, 전진하면서 방문한 노드들을 자식으로 하나씩 갖게 된다.

- Tree Edges: DFS Tree에 들어간 모든 엣지들
- Forward Edges: 조상에서 자손으로 연결되는 엣지 중 트리에 포함되지 않는 엣지들
- Back edges: 자손에서 자기보다 조상들 중 하나로 연결되는 엣지 (DFS 과정에서 이미 방문한 노드로 향하는 엣지)
- Cross Edges: 자손과 조상 관계가 아닌데 연결되는 엣지로 DFS Tree에는 이 엣지가 있으면 모순이 생겨서 존재하지 않는다.

11.3 Bipartite Graph Detection

Bipartite Graph는 이분 그래프로 두 개의 그룹으로 정점들을 나눴을 때 같은 집합끼리는 인접하지 않는 그래프를 의미한다. 이분 그래프인지 확인하는 방법은 DFS Tree를 만들었을 때 루트는 A, 레벨 1은 B, 레벨 2는 A, ... 이렇게 반복하면서 그룹 A B를 구분하는 것이다. 이때 서로 다른 그룹으로 가는 Back Edge는 존재할 수 있지만, 같은 그룹 내에서 이동하는 Back Edge는 존재하면 안된다.

12 Cut Vertex

12.1 Cut Vertex 알고리즘

Cut Vertex (절단점)는 제거할 경우 그래프가 끊어지게 되는 노드를 의미한다. 각 노드마다 노드를 지우고 그래프가 끊어졌는지를 확인하면 $O(mn)$ 의 시간이 걸리게 된다. 하지만 DFS를 돌리고 DFS Tree를 생성하면 $O(m+n)$ 만에 절단점을 찾을 수 있다.

Root Cut Vertex

DFS Tree의 루트가 2개 이상의 자식을 가지면 Cut Vertex임을 보이자. 루트에 연결된 서브트리들 중 두개를 각각 S1, S2라 하자. S1에 포함된 노드 중 하나를 x, S2에 포함된 노드 중 하나를 y라 하자. x에서 y로 가기 위해서 x는 S1을 벗어나야 한다.

1. Tree Edge만 사용할 경우: 이 경우 Root를 반드시 거칠 수 밖에 없다.
2. Tree Edge와 Back Edge도 사용할 경우: S1을 벗어나기 위해서는 S1보다 높은 조상으로 가야 하는데 가장 위의 조상이 루트이므로 반드시 Root를 거치게 된다.

따라서 루트는 Cut Vertex이다.

Non-Root Cut Vertex

DFS Tree에서 루트가 아닌 어떤 노드 t 가 Cut Vertex인지 확인하는 알고리즘을 알아보자. 모든 노드마다 $l(t)$ 를 계산하는데, $l(t)$ 는 아래 세가지 값들 중 minimum 값을 의미한다.

1. $Pre(t)$: DFS 방문 순서 번호 (Pre 번호로 조상-자손 관계만 따질 용도, 위로 갈수록 번호가 작다.)
2. t 와 연결된 Back Edge의 반대쪽 노드들의 $Pre()$ 값들 중 최솟값 (가장 높은 위치)
3. t 의 자손들의 $Pre()$ 값들 중 최솟값

쉽게 말해서, $l(t)$ 는 t 의 서브트리 내에서 Back Edge를 한번 타서 올라갈 수 있는 가장 높은 위치의 노드 번호를 의미한다. 노드 t 의 자식 u 가 존재하고, $l(u) \geq Pre(t)$ 이면 u 가 t 의 조상으로 가는 Back Edge가 없는 경우이므로, t 는 Cut Vertex이다.

Proof of $l(u) \geq Pre(t)$

t 의 자식 노드 u 의 서브트리를 S1, t 보다 조상인 어떤 노드를 y 라 하자. $u \rightarrow y$ 로 가려면 S1을 벗어나야 한다.

- Tree Edge만 사용할 경우: 이 경우 $u - t$ 엣지를 반드시 거치게 된다.
- Back Edge도 사용할 경우: $l(u) \geq Pre(t)$ 이므로 S1을 벗어나봤자 t 보다 아래에 있다. (u 에 t 의 조상으로 가는 Back Edge가 없다는 뜻)

12.2 Cut Vertex 알고리즘의 시간 복잡도

먼저 처음에 DFS Tree 생성하는데 $O(m+n)$ 이 걸린다.

1. $Pre(t)$: 처음에 DFS Tree 생성하면서 계산되어 있음
2. t 와 연결된 Back Edge의 반대쪽 노드들의 $Pre()$ 값들: 금방 계산 가능 (?)
3. t 의 자손들의 $Pre()$ 값들 중 최솟값: DFS를 Postorder 순으로 계산해서 한번 더 돌리면 된다.

이 과정에서 DFS를 한번 더 돌리기만 하므로 전체 시간 복잡도는 $O(m+n)$ 이다.

12.3 Cut Edge

Cut Edge는 제거했을때 그래프가 끊어지는 엣지를 말한다. 그래프 $G(m, n)$ 에서 모든 edge 마다 노드를 하나 추가한 그래프를 $G'(m+n, 2n)$ 이라 하자. 이 그래프에서 Cut Vertex 를 찾으면 그 노드를 추가한 edge가 Cut Edge이다. G' 에서 Cut vertex를 돌리는 것은 $O(m+n)$ 으로 동일하다.

12.4 BCC (Biconnected Component)

무향 그래프에서 정의되는 개념으로, Cut Vertex에 의해 끊어진 그래프들의 각 부분을 의미한다. BCC 내의 임의의 두 노드는 완전히 다른 (edge도 node도 겹치지 않는) 2개의 경로만 존재한다.

BCC를 찾기 위해서는 Cut Vertex 알고리즘을 사용하면 된다. Cut Vertex 알고리즘에서 아무리 올라가도 노드 t 까지밖에 못 가는 Subtree가 있으면 t 가 Cut vertex이었다. 이때 이 Subtree가 BCC 이다.

12.5 기출문제 Set 12

기출문제 12-1

Cut Vertex를 찾는 알고리즘이 있다고 가정할 때 Cut Edge를 찾는 알고리즘을 제시하라.
(시간복잡도, 정확성에 대한 증명은 필요 없다) (17기말)

13 DAG (Directed Acyclic Graph)

DAG는 사이클이 없는 방향 그래프를 의미한다. (한 점에서 출발해 자기 자신으로 돌아오는 경로가 존재하지 않는 경우)

13.1 Topological Sort

위상 정렬은 DAG의 노드들을 왼쪽에서 오른쪽 방향(엣지 방향이 모두 오른쪽) 으로 정렬하는 것을 의미한다. DAG에 사이클이 존재하지 않으면 알고리즘은 항상 성공한다.

DAG G 에서 $\text{indegree} = 0$ 인 노드 x 를 제거한 그래프를 G' 이라고 하자. (DAG에서 일부를 제거해도 DAG이다.) G' 에서 Topological Sort 알고리즘을 다시 반복하면 재귀적으로 정렬이 완료된다. x 의 indegree 가 0이기 때문에 x 에서 G' 사이에는 모두 왼쪽에서 오른쪽 방향 Edge만 존재한다. 즉 재귀적으로 위상 정렬이 된 DAG와 indegree 가 0인 노드와 연결하면 전체가 위상 정렬이 된다.

DFS를 한번 할때마다 노드를 하나 버릴 수 있으므로 DFS를 노드 개수만큼 반복하면 총 시간은 $O(mn)$ 이 걸린다.

성능 향상 (1) Event Queue

DFS를 한번 돌려서 각 노드들의 indegree 를 계산한다. 큐에 indegree 가 0인 노드들을 넣는다. 큐에서 노드 v 를 꺼내면 노드 v 에서 나가는 모든 엣지들을 제거하고 남은 노드들의 indegree 들을 업데이트 한다. indegree 가 0인 새로운 노드들이 나오면 다시 큐에 넣는다. 큐에서 꺼내진 노드들을 순서대로 출력하면 위상 정렬이 된다.

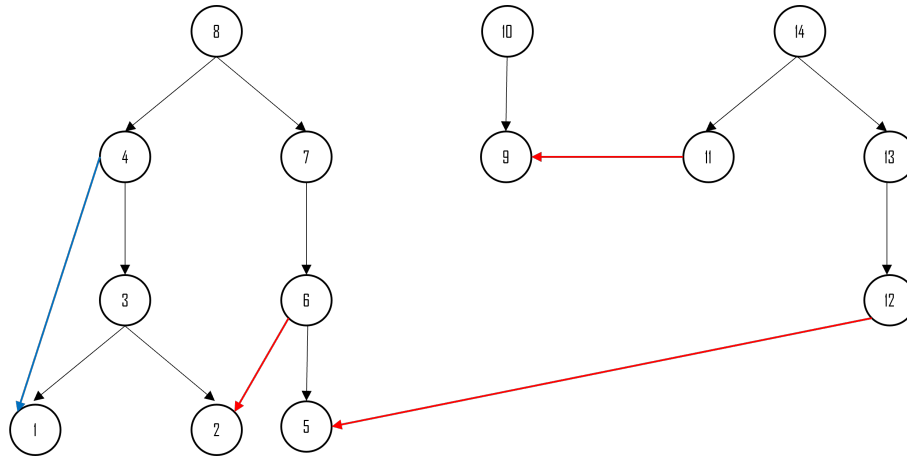
이 경우 처음에 DFS하는 과정이 $O(m + n)$, 각 노드들은 큐에 한번씩 들어가고 엣지들은 노드 출력 시 사용되므로 한번씩만 사용된다. 따라서 총 시간 복잡도는 $O(m + n)$ 이다.

성능 향상 (2) Post Numbers - 방향 그래프의 DFS Trees

DFS를 하면서 Post Number (떠날 때 번호 붙이기)를 붙이고, 이 번호 순서의 역순으로 노드들을 출력하면 그게 Topological Sort가 된다. 방향 그래프의 DFS Tree에는 아래와 같은 성질이 있다.

- Back Edge: 방향 그래프에서는 존재, DAG에는 존재하지 않는다.
- Tree, Forward, Cross Edge는 모두 Post번호가 큰 번호에서 작은 번호 방향으로만 존재한다. (왼쪽으로 갈수록, 아래로 갈수록 작아짐)

따라서 Post 번호의 역순이 Topological Sort이다.



13.2 SCC (Strongly Connected Component)

방향 그래프에서 임의의 두 정점에 대해 양 방향으로 가는 경로가 모두 있을 때 두 점은 같은 SCC에 속해 있다고 한다. 즉 SCC는 모든 노드가 양방향으로 Reachable한 노드의 Maximal Subset 이다. (예시. 사이클은 SCC) DFS Tree에서 여러 트리가 있을 때 하나의 트리에는 여러가지 SCC가 포함될 수 있지만, 하나의 SCC는 하나의 트리 안에서만 존재할 수 있다. 오른쪽에서 왼쪽으로 가는 Cross Edge는 존재하지만, 왼쪽에서 오른쪽으로 가는 Cross Edge가 존재하지 않기 때문이다.

Finding SCC

SuperNode는 SCC 안의 노드들 중 가장 큰 Post Number를 번호로 가지는 노드로, 노드 안에 SCC 그룹을 포함한다. SuperNode들로 Supergraph를 생성해보면 Topological Sort 순서대로 SCC들이 정렬되어 있다. 이를 이용하여 SCC를 구분해보자.

DFS Tree 안에 SCC가 여러개 있다고 가정하고, 이 SCC들을 구분하는 것이 목표이다. DFS를 돌려서 Post Number (떠날 때 번호 붙이기)를 붙이고, 모든 엣지의 방향을 반대로 뒤집는다. (이렇게 해도 SCC 자체는 변하지 않음) 가장 Post Number가 큰 노드를 찾자. 그 노드에서부터 DFS를 돌리면 하나의 SCC를 찾게 된다. 그 다음은 방문하지 않은 노드들 중 다음으로 큰 Post Number 노드에서부터 위의 과정을 반복하면 된다.

13.3 Shortest Longest Paths on DAG

topological sort가 된 DAG에서는 Shortest, Longest Path를 찾는 것이 쉬워진다. 간선에 음수가 있어도 구할 수 있다. SSSP 문제는 정렬된 노드 순서대로 다익스트라처럼 가중치를 업데이트 시켜 주면 되고, 입구에서 출구로 가는 SP, LP 문제는 DP와 유사한 방식으로 풀 수 있다.

13.4 DFS Tree 총정리

1. 무향 그래프의 DFS Tree: Tree edge, Back edge 존재
2. 방향 그래프의 DFS Tree: Tree edge, Forward edge, Back edge, Cross edge 존재. 이때 Back edge는 사이클을 생성한다. 하나의 트리에 여러 개의 SCC가 들어 있는 경우를 생각해보자. 트리에서 상대적으로 위쪽에 있는 SCC에서 아래쪽에 있는 SCC 방향으로 Forward edge가 존재할 수 있지만, 아래에서 위쪽 방향으로 가는 Back edge는 존재할 수 없다. (하지만 하나의 SCC 안에서는 위쪽 방향으로 가는 Back edge는 존재 가능하다.) Cross edge도 오른쪽에서 왼쪽 방향으로만 가능하지 왼쪽에서 오른쪽 방향의 SCC로는 이동할 수 없다. (DFS의 Post 번호 기준 큰 번호에서 작은 번호 방향으로만 이동할 수 있다.)
3. DAG의 DFS Tree: Tree edge, Forward edge, Cross edge 존재 / Back edge 는 존재하지 않음 (사이클 생성)

13.5 기출문제 Set 13

기출문제 13-1

Directed Graph 에서 DFS의 결과로 얻어지는 Tree T에서 가능한 Back Edge의 형태들을 제시하고 그 형태들만이 가능하다는 점을 증명하라. (14기말)

기출문제 13-2

Directed Acyclic Graph를 DFS를 통해 트리로 만든다고 가정하자. (17기말)

- (a) 여기서 생기는 Back Edge는 어떻게 생겼고 왜 그렇게 생겼는지를 증명하라.
- (b) (a)를 이용하여 DAG에 in-degree가 0인 노드가 반드시 존재함을 증명하라.

14 Exhaustive Search

14.1 Backtracking: N-queen

N queen 문제는 $n \times n$ 크기의 체스판에 n개의 퀸을 서로 공격하지 않도록 놓는 경우의 수를 구하는 문제이다. 한 row에 퀸이 하나만 있을 수 있으므로 최소 시간 n^n 이 걸린다. 근데 내려갈수록 가능한 범위가 점점 제한되는데 이걸 시간 재기가 어렵다.

14.2 Pseudo-polynomial

Pseudo-polynomial 문제들은 n의 크기에 따라 걸리는 시간이 달라지는 유형을 의미하는데 간단히만 보고 넘어가자.

1. 동전 교환: n개의 동전 단위 C[i]가 있고, 각 동전의 개수는 무한할 때 W원을 동전으로 바꾸기 위해 필요한 최소 동전의 개수를 구하는 문제 $\rightarrow D[i] = \min(D[i - C[j]] + 1), Ans = D[w]$
2. 막대 자르기: 길이 i인 막대기의 가치 V[i]가 입력으로 주어졌을 때, 길이 N인 막대기를 잘 나누어서 최대 가치로 나누는 문제 $\rightarrow D[i] = \max(D[i - j] + V[j])(j : 1 \sim i)$
3. 합 분해: 0부터 n까지의 정수 k개를 더해서 합이 n이 되는 경우의 수를 구하는 문제 $\rightarrow D[i][j] = \sum(D[i - 1][j - k])(k : 0 \sim j) = D[i - 1][j] + D[i][j - 1]$

14.3 State Space

15 puzzle에서 각 상태들을 하나의 노드로 가지면 각 상태들간에 엣지를 넣어 연결 요소, 그래프 등 여러 형태로 나타낼 수 있다. 15 puzzle에는 모든 경우의 수들을 그래프로 표현하면 2개의 Connected Components 가 존재한다. 하나는 해가 존재하는 CC, 나머지 하나는 해가 존재하지 않는 CC 이다. 어떤 상태에서 임의의 두 조각의 위치를 변경하면 한번 변경할때마다 다른쪽 Connected Component 로 이동하게 된다.

14.4 Traveling Salesman Problem, Cutoff

TSP 문제는 모든 노드를 한번씩만 최소한의 비용으로 방문하는 문제이다. 방문할 수 있는 경우들을 State Tree로 나타냈을 때 cutoff 를 어떻게 할 수 있을지 알아보자. 시작 노드를 S, 방문한 노드 집합을 V, 아직 방문하지 않는 노드 집합을 X라 하자.

1. S가 X의 노드들을 잇는 엣지 weight 중 최솟값 (돌아오는 마지막 edge weight 의미)
2. 제일 최근에 V 집합에 추가된 노드와 X의 노드들을 잇는 엣지 weight 중 최솟값
3. X의 각 노드 i들에 대해서 [X의 다른 노드, S, 최근에 V집합에 추가된 노드]들을 잇는 엣지 weight 중 최솟값 2개

Cut off 기준은 위의 값들에 대하여 $(1 + 2 + 3)/2$ 개이다. 즉 이 개수만큼은 최소한 써야 한다는 뜻이다.

15 Randomized Algorithm

15.1 Randomized Quick Sort

기존에는 Quick Sort를 하면 어떤 데이터는 유리하고 어떤 데이터는 불리하게 작용하는 경우가 있을 수 있었다. 만약 Quick Sort에서 Pivot을 랜덤하게 뽑을 경우 특정한 고정된 입력에 대해서도 평균적인 성능이 나오도록 할 수 있다.

15.2 Linked list in an Array: Las Vegas

n 개의 정렬된 연결 리스트에서 특정 값 X 를 찾는 문제이다. 알고리즘은 비교적 간단하다. $\sqrt{n}$ 개의 랜덤 숫자를 선택하고 그 중에서 X 보다 작은 숫자들 중 가장 큰 숫자를 뽑고, 그 숫자들로부터 링크를 따라서 탐색하면 된다. 시간 복잡도는 k 가 처음에 랜덤하게 뽑은 멤버들의 개수라고 할 때 $O(k + f(n, k))$ 가 걸린다. $f(n, k)$ 는 선택된 숫자와 정답으로부터의 거리가 1, 2, ..., $n-1$ 일 확률들을 더한 것인데 자세한 것은 생략한다. 식을 정리하면 $O(k + n/(k+1))$ 시간이 나오는데, k 가 $\sqrt{n}$ 일때 시간 복잡도 $O(\sqrt{n})$ 만에 값을 찾을 수 있다.

15.3 Primality Test: Monte Carlo

무슨 소리인지 하나도 모르겠다..... 시험 나오면 스킵

15.4 Lowest Common Ancestor

모든 노드에 레벨 정보가 기록된 트리에서 두 노드 a, b 의 최소 공통 조상 x 를 찾아보자.

Preprocessing

먼저 각 노드들이 1, 2, 4, 8, 16, ..., 2^j 칸 위의 조상의 번호를 저장하는 과정이 필요하다. $A[i][j]$ 는 i 번 노드의 2^j 칸 위의 조상의 번호를 저장한다. 이때 j 의 값은 0부터 $\log n$ 까지이다.

- Base: $A[i][0]$: 나의 부모의 ID 값
- Step: $A[i][j+1] = A[A[i][j]][j]$: 모든 노드가 2^j 칸 위의 ID를 안다고 가정하면 2^{j+1} 칸 위의 ID는 2^j 칸 위로 올라간 뒤 ($A[i][j]$ 로 온 뒤) 이 노드에서 한번 더 2^j 칸을 올라가면 된다.

Finding LCA

위에서 생성한 배열 A 를 이용하여 노드 a 와 b 의 LCA를 찾아보자.

1. a 를 b 와 같은 레벨로 올리자. 이때 레벨의 이동은 2의 거듭제곱꼴로 이루어진다. 예를 들어 a 가 b 보다 11칸 아래에 있으면, $11 = 1011_{(2)}$ 이므로 8칸, 2칸, 1칸 위로 올라가야 된다.

- a) $a \rightarrow a_1 = A[a][3]$
- b) $a_1 \rightarrow a_2 = A[a_1][1]$
- c) $a_2 \rightarrow a' = A[a_2][0]$

이렇게 a 를 올려서 b 와 같은 높이에 오는 노드를 a' 이라 하자.

2. 이제 a' , b 모두 같은 레벨에서 위로 점프를 하면서 공통 조상 여부를 판단하자. $A[a'][j]$, $A[b][j]$ 의 값이 같은지 다른지 판단한다. 만약 같으면 j 를 하나 감소시키고, 다르면 그 노드에서 정지한다. 정지한 노드가 $j = t$ 일때라고 가정하면 노드 t 에서부터 다시 위의 과정을 반복해서 $A[A[a'][t]][j]$, $A[A[b][t]][j]$ 의 값이 같은지 다른지를 판단하면 된다. 이때 j 의 초기값은 $\log n$ 이고 j 가 0이 되면 알고리즘을 종료한다.

Preprocessing 단계는 $O(n \log n)$ 이 소요되고, 그걸로 LCA를 찾는 과정은 $O(n)$ 이 소요된다.

15.5 Boruvka's MST Algorithm: Randomized MST

Boruvka의 알고리즘은 각 노드마다 weight가 가장 작은 엣지를 선택하고, Connected Component를 하나의 노드로 합치는 과정을 전체에서 하나의 노드만 남을때까지 반복하는 것이다. 이때 각 노드마다 선택한 엣지는 MST에 들어가게 되고, 각 라운드마다 노드의 개수가 $1/2$ 이 된다.

이제 Randomized MST를 찾아보자. 그래프 G 에서 Boruvka 알고리즘을 3번 돌려서 노드의 개수를 $1/8$ 로 만든 그래프를 G_1 이라 하자. G_1 에서 엣지를 50% 확률로 선택해서 엣지의 개수를 $1/2$ 로 줄인다. 전체 알고리즘을 재귀로 돌린 결과를 F 라 하자. (MSF) F 에서 Heavy Edge들을 제거하고, 다시 전체 알고리즘을 재귀로 돌린 뒤 그 결과를 G_2 이라 하자. 그럼 최종 정답은 $G_1 \cup G_2$ 이다.

Heavy Edge란 그 엣지를 포함한 사이클 내에서 그 엣지를 제외한 나머지 엣지들의 weight들 보다 큰 엣지를 의미한다. Heavy Edge를 찾기 위해서 edge weight에 minimum 값들을 넣으면서 LCA를 하면 제일 작은 edge weight를 알 수 있다.

15.6 기출문제 Set 15

기출문제 15-1

Randomized QuickSort 알고리즘을 제시하라. (14기말)

기출문제 15-2

잘 정렬된 자연수 배열 $A[], B[]$ 에서 각각 한개씩 뽑아 둘이 합친 값이 X 인지를 구하는 알고리즘을 제시하고 정확성과 시간복잡도를 증명하라. (17기말)